# Assignment_01_CO_02

**Q1** **Check whether a number is an Emirp number** 5 marks

Topics to use: Classes, objects, member functions, loops, digit reversal, prime checking, logical validation.

Question: An Emirp number is a prime number whose reversed number is a different prime number.

Example: 13
- 13 is prime.
- Reversed number = 31.
- 31 is also prime and differs from 13.
- Therefore, 13 is an Emirp number.

Test Cases:
1.13->Emirp number
2.23->Not an Emirp number
3.1->Invalid input

Your Code

```cpp
#include <iostream>
using namespace std;

class Emirp {
public:
    bool isPrime(int n) {
        if (n < 2)
            return false;

        for (int i = 2; i * i <= n; i++) {
            if (n % i == 0)
                return false;
        }
        return true;
    }

    int reverseNumber(int n) {
        int rev = 0;

        while (n > 0) {
            rev = rev * 10 + n % 10;
            n = n / 10;
        }

        return rev;
    }

    void check(int n) {
        if (n <= 1) {
            cout << "Invalid input";
            return;
        }

        int rev = reverseNumber(n);

        if (isPrime(n) && isPrime(rev) && n != rev)
            cout << "Emirp number";
        else
            cout << "Not an Emirp number";
    }
};

int main() {
    int n;
    cin >> n;

    Emirp obj;

    if (n <= 1)
        cout << "Invalid input";
    else
        obj.check(n);

    return 0;
}
```

Submitted

Input

```
1.13
```

Output

```
Invalid input
```

Visible Test Cases

Test Case 1 ✓

Test Case 2 ✓

Test Case 3 ✓

5.00 / 5 marks

**Q2** **Check whether a number is a Circular Prime** 5 marks

Topics to use: Classes, objects, member functions, loops, prime checking, digit rotation, arrays, logical validation.

Question: A Circular Prime is a prime number for which every number formed by rotating its digits is also prime.

Example: 197
- Original number: 197 → prime.
- Rotate left: 971 → prime.
- Rotate left again: 719 → prime.
- Therefore, 197 is a Circular Prime.

Test Cases:
1.197->Circular Prime
2.101->Not a Circular Prime
3.2->Circular Prime

Your Code

```cpp
#include <iostream>
using namespace std;

class CircularPrime {
public:
    bool isPrime(int n) {
        if (n < 2)
            return false;

        for (int i = 2; i * i <= n; i++) {
            if (n % i == 0)
                return false;
        }

        return true;
    }

    void check(int n) {
        if (n < 2) {
            cout << "Circular Prime";
            return;
        }

        int temp = n;
        int digits = 0;

        while (temp > 0) {
            digits++;
            temp /= 10;
        }

        int power = 1;

        for (int i = 1; i < digits; i++) {
            power *= 10;
        }

        temp = n;

        for (int i = 0; i < digits; i++) {
            if (!isPrime(temp)) {
                cout << "Not a Circular Prime";
                return;
            }

            int last = temp / power;
            temp = (temp % power) * 10 + last;
        }

        cout << "Circular Prime";
    }
};

int main() {
    int n;
    cin >> n;

    CircularPrime obj;
    obj.check(n);

    return 0;
}
```

Input

```
1.197
```

Output

```
Circular Prime
```

Visible Test Cases

Test Case 1 ✓
Test Case 2 ✓
Test Case 3 ✓

5.00 / 5 marks

Submitted

## Q3 Check whether a number is a Kaprekar number

5 marks

Topics to use: Classes, objects, member functions, loops, digit counting, arithmetic operations, conditional logic.

Question: A positive number is a Kaprekar number if its square can be split into two parts such that the sum of those parts equals the original number. The right part must contain as many digits as the original number.

Example: 45
- Square = \(45^2 = 2025\)
- Split into 20 and 25.
- Sum = \(20 + 25 = 45\).

Test Cases:
1.9->Kaprekar number
2.45->Kaprekar number
3.10->Not a Kaprekar number

Your Code

```cpp
#include <iostream>
using namespace std;

class Kaprekar {
public:
    void check(int n) {
        if (n <= 0) {
            cout << "Not a Kaprekar number";
            return;
        }

        long long square = (long long)n * n;
        long long temp = n;
        long long power = 1;

        while (temp > 0) {
            power *= 10;
            temp /= 10;
        }

        long long right = square % power;
        long long left = square / power;

        if (left + right == n)
            cout << "Kaprekar number";
        else
            cout << "Not a Kaprekar number";
    }
};

int main() {
    int n;
    cin >> n;

    Kaprekar obj;
    obj.check(n);

    return 0;
}
```

Input

```
1.9
```

Output

```
Kaprekar number
```

Visible Test Cases

- Test Case 1 ✓
- Test Case 2 ✓
- Test Case 3 ✓

5.00 / 5 marks

Submitted

## Q4 Sphenic Number

5 marks

Topics: Classes and objects, access specifiers, constructors, member functions, loops, prime factorization, and arrays of objects.
Write a C++ program using a class named NumberAnalyzer to check whether a positive integer is a Sphenic Number. A Sphenic Number is a number that can be expressed as the product of exactly three distinct prime numbers, each appearing only once.

The program must identify the prime factors, count them, and verify that exactly three distinct prime factors multiply to the original number.

Example:

Input: 30
\[
30=2\times3\times5
\]

Since 2, 3, and 5 are three distinct primes, 30 is a Sphenic Number.

Test Cases:
1.30->Sphenic
2.42->Sphenic
3.210->Not Sphenic

Your Code

```cpp
#include <iostream>
using namespace std;

class NumberAnalyzer {
public:
    void check(int n) {
        if (n <= 0) {
            cout << "Not Sphenic";
            return;
        }

        int count = 0;
        int temp = n;

        for (int i = 2; i <= temp / i; i++) {
            if (temp % i == 0) {
                int exponent = 0;

                while (temp % i == 0) {
                    temp /= i;
                    exponent++;
                }

                if (exponent != 1) {
                    cout << "Not Sphenic";
                    return;
                }

                count++;
            }
        }

        if (temp > 1)
            count++;

        if (count == 3)
            cout << "Sphenic";
        else
            cout << "Not Sphenic";
    }
};

int main() {
    int n;
    cin >> n;

    NumberAnalyzer obj;
    obj.check(n);

    return 0;
}
```

Submitted

Input

stdin input

Output

Visible Test Cases

Test Case 1

Test Case 2

Test Case 3

5.00 / 5 marks

## Q5 Check whether a number is a Trimorphic number

5 marks

Topics to use: Classes, objects, private data members, member functions, loops, arithmetic operations, remainder operator.

Question: A number is called a Trimorphic number if its cube ends with the original number.

Example: 24
- Cube = \(24^3 = 13824\)
- The last two digits are 24.
- Therefore, 24 is a Trimorphic number.

Test Cases:
1.24->Trimorphic number
2.5->Trimorphic number
3.7->Not a Trimorphic number

Your Code

```cpp
#include <iostream>
using namespace std;

class Trimorphic {
private:
    long long n;

public:
    Trimorphic(long long num) {
        n = num;
    }

    void check() {
        if (n < 0) {
            cout << "Not a Trimorphic number";
            return;
        }

        long long cube = n * n * n;
        long long temp = n;
        long long power = 10;

        while (temp >= 10) {
            power *= 10;
            temp /= 10;
        }

        if (cube % power == n)
            cout << "Trimorphic number";
        else
            cout << "Not a Trimorphic number";
    }
};

int main() {
    long long n;
    cin >> n;

    Trimorphic obj(n);
    obj.check();

    return 0;
}
```

Submitted

Input

```
1.24
```

Output

```
Trimorphic number
```

Visible Test Cases

- Test Case 1 ✓
- Test Case 2 ✓
- Test Case 3 ✓

5.00 / 5 marks

**Q6** **Check whether a number is a Keith number** 5 marks

Topics to use: Classes, objects, member functions, loops, arrays, digit extraction, sequence generation.

Question: A Keith number is a positive integer whose digits form the beginning of a sequence. Each subsequent term is the sum of the previous terms, using as many terms as the original number has digits. If the sequence reaches the original number, it is a Keith number.

Example: 197

Starting digits: 1, 9, 7
- \(1+9+7=17\)
- \(9+7+17=33\)
- \(7+17+33=57\)
- \(17+33+57=107\)
- \(33+57+107=197\)

Therefore, 197 is a Keith number.

Test Cases:
1.197->Keith number
2.742->Keith number
3.7->Not a Keith number

Your Code

```cpp
#include <iostream>
using namespace std;

class KeithNumber {
private:
    int n;

public:
    KeithNumber(int num) {
        n = num;
    }

    void check() {
        if (n < 10) {
            cout << "Not a Keith number";
            return;
        }

        int digits[20];
        int count = 0;
        int temp = n;

        while (temp > 0) {
            digits[count] = temp % 10;
            count++;
            temp /= 10;
        }

        for (int i = 0; i < count / 2; i++) {
            int t = digits[i];
            digits[i] = digits[count - 1 - i];
            digits[count - 1 - i] = t;
        }

        int sum = 0;

        while (true) {
            sum = 0;

            for (int i = 0; i < count; i++) {
                sum += digits[i];
            }

            if (sum == n) {
                cout << "Keith number";
                return;
            }

            if (sum > n) {
                cout << "Not a Keith number";
                return;
            }

            for (int i = 0; i < count - 1; i++) {
                digits[i] = digits[i + 1];
            }

            digits[count - 1] = sum;
        }
    }
};

int main() {
    int n;
    cin >> n;
```

Input

```
1.197
```

Output

```
Not a Keith number
```

Visible Test Cases

Test Case 1 ✓
Test Case 2 ✓
Test Case 3 ✓

5.00 / 5 marks